第 11 章　基于编码-解码器架构的鲁棒图像数字水印算法

本实验基于论文 MBRS: Enhancing Robustness of DNN-based Watermarking by Mini-Batch of Real and Simulated JPEG Compression 设计，实现了一个基于编码-解码器架构的、具有强鲁棒性（robustness）的数字水印系统。

11.1　实验目的

1. 理解编码-解码器架构（Encoder-Decoder）的原理与实现
2. 通过对比 MBRS 的 SE Block 通道注意力和 Transformer 的注意力层，加深对“注意力机制”的理解
3. 通过学习 MBRS 的扩散块（Diffusion Block），加深对卷积神经网络中“感受野”概念的理解
4. 以 JPEG 图像的数字水印算法为引例，学习神经网络对全零梯度损失函数的处理思想
5. 掌握数字水印技术（Watermarking）的概念和原理

11.2　实验原理

11.2.1　数字水印技术

数字水印（Watermarking）是一种将特定信息嵌入到数字媒体（如图像、音频或视频）中的信息隐藏技术，其目标是在不影响媒体正常使用的前提下，实现版权保护、身份认证或篡改检测等功能。即便图片被部分截取、压缩、调节，乃至印成照片后再拍摄（Print-and-Scan），它都能保护其中的版权信息，具有良好的不可见性与鲁棒性。与之类似的一项技术是图片隐写术（Image Steganography），后者的目标是不可感知性（Imperceptibility），即在图片中藏匿不希望为他人所发现的信息。

数字水印的历史可以追溯到 1994 年。早期的研究者尝试修改图像像素的最低有效位

（Least Significant Bit，LSB），但这种方法十分脆弱，乃至无法经受简单的亮度调整或格式转换。后来出现了变换域方法，即将图片通过离散余弦变换（Discrete Cosine Transform，DCT）或离散小波变换（Discrete Wavelet Transform，DWT）转换到频率域，在这些频率系数里隐藏版权信息，可以极大提高鲁棒性。随着深度学习的发展，现代基于深度学习的数字水印算法渐趋成熟，它们主要利用自编码器（Auto-encoder）架构，这是一种特殊的编码-解码器架构。其中，编码器（Encoder）负责嵌入水印，解码器（Decoder）负责提取水印，中间加一个“噪声层”（Noise Layer）来模拟现实中的各种破坏。

(a) (b)

Fig.1. (a) Original image (b) Watermark

(a) (b)

Fig.2. (a) Watermarked image (b) Retrieved image

图 11.1　DWT-DCT-SVD 效果图示，图摘自原论文。上图展示了原图和待嵌入水印，下图则展示了嵌入后的水印图和提取出的水印

数字水印技术中所要嵌入的水印，一般指固定长度的 0/1 随机比特串。基于深度学习的数字水印方案目前最大的问题是，为了保证鲁棒性和模型收敛，嵌入的水印长度不可过长。对于 128×128 的图片，可嵌入比特串的长度一般为 30 bits 到 64 bits 之间，而对于 256×256 的图片，则可以达到 256 bits。更长的比特串会导致模型的鲁棒性受损，同时也不易训练。

相比之下，传统方案在不考虑极致鲁棒性的情况时，通常能提供更高的嵌入容量。例如基于 DWT-DCT-SVD 方案（见图 11.1），已经能够实现在 256×256 的灰度图中嵌入 128×128 的灰度图水印（展平为一维序列则是长度为 $8 \times 128 \times 128$ 的比特串）。

深度学习方案相较于传统方案，提供的是不同的价值维度，例如：自适应嵌入（智能的内容感知）、端到端的对抗学习机制、强鲁棒性（在极端攻击下的强生存能力）等。传统方案至今仍具研究价值，且传统变换理论与深度神经网络的深度融合被公认为未来数字水印领域的重要突破方向。

11.2.2　编码-解码器架构

编码-解码器架构最早在自然语言处理（Natural Language Processing，NLP）中被大量使用，是一种序列到序列（Sequence-to-Sequence，Seq2Seq）模型，即以序列数据作为输入和输出。它的目标是将一个领域的输入 X 转换为另一个领域的输出 Y。一个典型的例子是机器翻译，如 X 为中文“我爱你”，而 Y 为英文“I Love You”，这里的 X 和 Y 属于语言不同，且长度不同的序列数据。

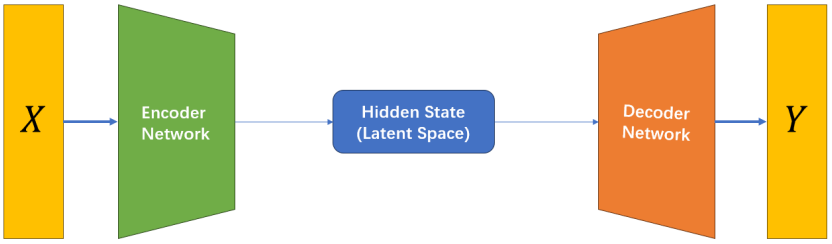

图 11.2　编码-解码器架构

如图 11.2 所示，编码-解码器架构由三部分组成：

- **编码器（Encoder）**：一个神经网络，其将序列输入 X 转化为固定长度的语义向量（嵌入，Embedding）。
- **隐藏状态（Hidden State）**：编码器输出的语义向量，可以理解为神经网络的隐藏层，其挖掘并记忆了输入序列的深层特征和语义信息，且将作为解码器的输入。
- **解码器（Decoder）**：一个神经网络，其将语义向量转变为长度可变的序列输出 Y。

编码-解码器架构后被证实是一种十分泛用的架构，因为 X 和 Y 可以是不同模态的数据。比如，X 可以是一个图片的像素矩阵，而 Y 则是一个文字序列，只需要对编码器和解码器的网络结构做调整，它就可以兼容不同模态的数据。以编码-解码器架构为基础的学习一

般是有监督的（Supervised Learning），需要成对的数据作为训练集。编码-解码器架构的直接典型应用如机器翻译、图像描述等。

而相应的，如果 X 和 Y 属于同一领域的数据（不止要求同模态，更要求同格式，如相同长度的序列），则可以通过自监督学习（Self-supervised Learning）来提取数据的深层特征与语义信息，这样的架构便是自编码器架构（Autoencoder），它的最终目的是学习数据的低维压缩表示，因此语义向量的维数总是小于输入和输出的维数。自编码器架构的典型案例如图像去噪、数据降维、异常检测等。

在鲁棒数字水印研究中，这种架构被进一步拓展：虽然它在形式上类似于自编码器（希望输入和输出在图像域上保持高度视觉相似），但其核心目标不再仅仅是数据的低维重构，而是秘密信息的隐藏与抗损提取。为了实现这一目标，研究者在编码器和解码器之间引入了噪声层（Noise Layer），其架构可用图 11.3 描述。噪声层的主要功能是模拟现实世界中图像可能遭受的各种“攻击”或处理过程。在带噪声层的端到端训练中，数据流发生了质的变化：编码器生成的含水印图像 I_{EN} 并不直接交给解码器，而是先穿过噪声层，被转化为“受损”的加噪图像 I_{NO}。以 MBRS 为例，噪声层会通过真实或模拟的 JPEG 压缩算法对图像像素进行量化破坏。此时，解码器的任务难度剧增：它必须学会忽略那些由于压缩产生的方块效应和纹理损失，从受损的 I_{NO} 中恢复出原始的水印比特串 M。

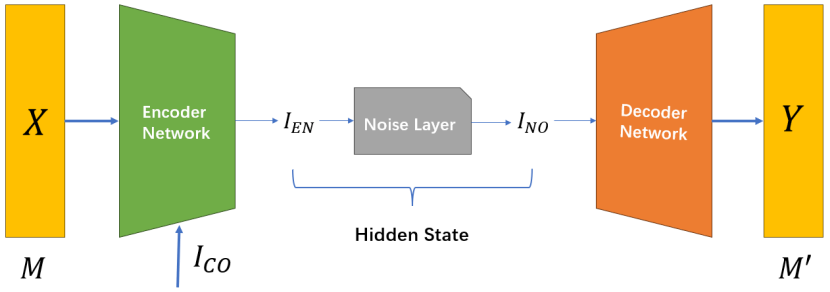

图 11.3　数字水印语境下的编码-噪声层-解码器架构

11.2.3　JPEG 图像压缩算法的全零梯度缺陷

JPEG 压缩算法的原理

JPEG 是一种常用的图像压缩算法，利用它压缩生成的 .jpg 和 .jpeg 文件通常能相比原文件减少大量存储占用。它的主要步骤如下：

1. 将图片从 RGB 红绿蓝空间转换到 YCbCr 亮度色度空间，并舍去人眼不敏感的颜色信息（色度下采样，Chroma Subsampling）
2. 将图像进行切块，方便后续的离散余弦变换处理局部特征
3. 利用离散余弦变换将空间域的像素值转变为频域的系数。对于划分大小为 $N \times N$ 的

图像块,其变换公式为:

$$F(u,v)=\frac{2}{N}C(u)C(v)\sum_{i=0}^{N-1}\sum_{j=0}^{N-1}f(i,j)\cos\left[\frac{(2i+1)u\pi}{2N}\right]\cos\left[\frac{(2j+1)v\pi}{2N}\right]$$

式中，$f(i,j)$ 为输入的空间域数据，即坐标 (i,j) 处的像素亮度值；$F(u,v)$ 为输出的频率域系数，代表频率为 (u,v)（水平频率、垂直频率）的分量在原图中的权重；$C(u),C(v)$ 为补偿缩放因子,以保证该变换是正交的:

$$C(k)=\begin{cases}\frac{1}{\sqrt{2}}, & k=0\\ 1, & k>0\end{cases}$$

4. 量化（Quantization）。这一步是有损压缩真正发生的地方,其将 DCT 的系数矩阵除以一个预定义的量化表并取整

$$B_{u,v}=\text{round}\left(\frac{F_{u,v}}{Q_{u,v}}\right)$$

其中 $Q_{u,v}$ 称为质量因子（Quality Factor）,用以控制压缩的强度;round(·) 为四舍五入函数。由于经过离散余弦变换后,大部分高频系数会被除成 0.1, 0.2 之类的数字,因此它们在经由取整后,都会变成 0,这便是其有损压缩的原理:把不重要的细节变成零。

5. 熵编码。利用哈夫曼编码等技术压缩剩下的非零值,生成最终文件。

量化公式的全零梯度缺陷

一个鲁棒的数字水印系统,其添加的数字水印应当要在有损压缩后仍然保持完整。因此,以编码-解码器架构为基础的现代数字水印系统必须考虑 JPEG 压缩的数学过程,并将其纳入训练架构的范畴。然而,在网络架构里加入压缩过程便存在损失函数梯度为零的问题。一个考虑了 JPEG 压缩的,基于编码-解码器架构的数字水印系统简化流程可以如下表示:

$$M\xrightarrow{\text{Encoder }\theta_E}I_{EN}\xrightarrow{\text{JPEG}}I_{NO}\xrightarrow{\text{Decoder }\theta_D}M'$$

需要被嵌入的信息 M 经由编码器嵌入到图像 I_{EN}（ENcoded Image）中,通过 JPEG 压缩得到带有噪声的图像 I_{NO}（NOised Image）,其最后通过解码器得到提取后的信息 M'。损失函数 L_D 则可通过 M 和 M' 之间的误差定义。

为了训练编码器,需要计算损失函数 L_D 对编码器参数 θ_E 的导数:

$$\frac{\partial L_D}{\partial\theta_E}=\frac{\partial L_D}{\partial M'}\cdot\frac{\partial M'}{\partial I_{NO}}\cdot\boxed{\frac{\partial I_{NO}}{\partial I_{EN}}}\cdot\frac{\partial I_{EN}}{\partial\theta_E}$$

等号右侧框选部分的这一项包含了噪声图像对编码图像的导数。该求导过程涉及 JPEG 的压缩,由于量化过程函数中,round(·) 的导数在绝大多数点处恒为 0,而在其它点处不可导,这意味着传递到编码器的梯度几乎无法对参数做出有效更新。为了解决全零梯度的问题,目前有两种主流方案:

1. **OEDS（One-stage End-to-end with Simulated JPEG）**：即模拟 JPEG，旨在用一个可导的数学公式去模拟 JPEG 的压缩效果。
2. **TSR（Two-stage Separable）**：将训练过程分为两步，第一阶段先采用不经过 JPEG 压缩的无噪声训练，然后在第二阶段，引入 JPEG 压缩所带来的噪声，固定编码器，只训练解码器来适应真实的 JPEG。

这两种方案各有缺陷，前者终归只是模拟，模型在遇到真实的 JPEG 时仍然会出错；后者则只训练了解码器，而没有对编码器进行优化。这两种方案的流程如图 11.4 所示。

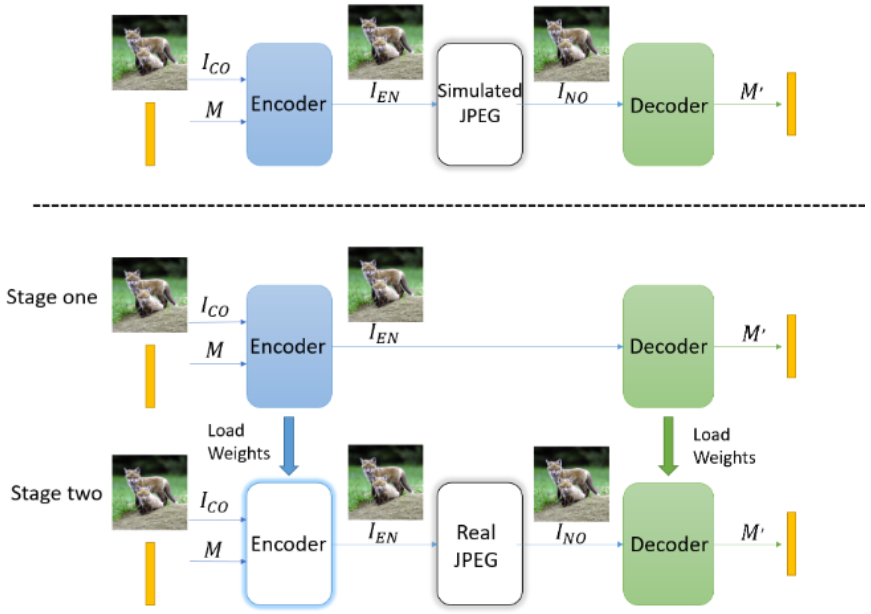

图 11.4　两种主流方案，上为 OEDS，下为 TSR。图摘自原论文

11.2.4　MBRS: 小批量混合真实与模拟数据训练

MBRS 的全称是 Mini-Batch of Real and Simulated（JPEG），即小批量混合真实与模拟 JPEG。它的核心思想十分简单：既然模拟 JPEG 效果不好，真实 JPEG 又存在全零梯度问题，那么便将它们混合在一起进行训练。

噪声层切换逻辑

在每一个训练迭代（Mini-batch）中，MBRS 会随机从以下三种模式中选一种作为当前的噪声层：

- **真实 JPEG**：直接调用标准的 JPEG 压缩算法。虽然会出现全零梯度的问题，但解码器依然能通过更新自己的参数来提升提取能力。

- **模拟 JPEG**：采用主流的模拟 JPEG 算法。MBRS 对比并使用两种方案：由 Zhu 等人提出的 JPEG-Mask 和 Shin 等人提出的 JPEG-SS 限于篇幅且并非本次实验的重点，故不对它们展开叙述。
- **恒等映射/无噪声**：不添加任何噪声。

MBRS 的噪声层切换逻辑可用图 11.5 表示。值得注意的是，即便在当前训练迭代中选择了真实 JPEG，导致全零梯度的出现，编码器的参数仍然可以通过动量优化器（Momentum-based Optimizers，比如 Adam Optimizer）在先前模拟 JPEG 或恒等映射中累计的动量来更新编码器参数。这一过程的具体理解将留作思考题。

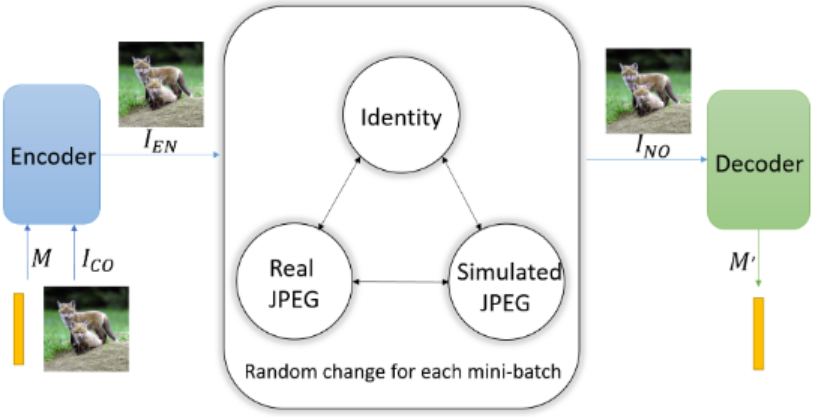

图 11.5　MBRS 的噪声层切换逻辑。图摘自原论文

SE Block：通道注意力机制

SE Block，全称为 Squeeze-and-Excitation Block，由 Hu 等人在论文 Squeeze-and-Excitation Networks 中提出，它的核心逻辑是：并非所有的特征通道都同样重要。在卷积神经网络中，每一层都会产生许多通道，有的通道可能学习的是图像的边缘信息，有的可能学到了图像的色彩信息，而有的可能只是单纯的噪声。传统的卷积层对所有通道都进行加和，而 SE Block 则为每个通道都添加了权重，来区分每个通道的重要性。

具体来说，对于一个大小为 $H \times W \times C$ 的特征图，SE Block 所做的操作可总结为三步：

1. **压缩（Squeeze）**。其通过全局平均池化将每一个大小为 $H \times W$ 的通道挤压为一个单一的数值，如此便得到了一个长度为 C 的向量，这个向量包含了每个通道的全局信息摘要。
2. **激发（Excitation）**。根据第一步所得到的摘要，判断每个通道的重要性。其先通过一个全连接层，将向量的维度由 C 降至 C/r；接着，通过 ReLU 激活层，再经由全连接层升回 C 维向量；最后通过 Sigmoid 激活层，将数值归一化到 $0 \sim 1$ 之间。如此便得到了直观的权重值。

3. **缩放（Scale）**。将得到的权重向量分别乘以原始特征图中对应的通道，便可以加强重要的通道，抑制无用的通道。

SE Block 的架构图如图 11.6 所示。

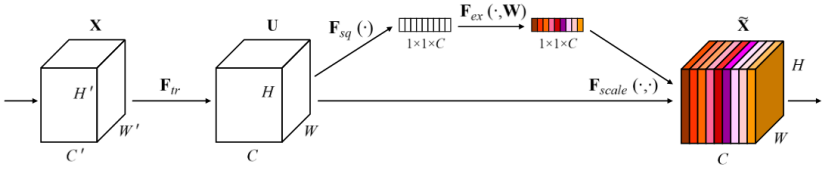

图 11.6　一个 SE Block。图摘自原论文

系统架构

MBRS 的架构如图 11.7 所示，其由五个部分组成：消息处理器（Message Processor）、编码器（Encoder）、解码器（Decoder）、JPEG 噪声层（JPEG Noise Layer），以及对抗辨别器（Adversary Discriminator）。

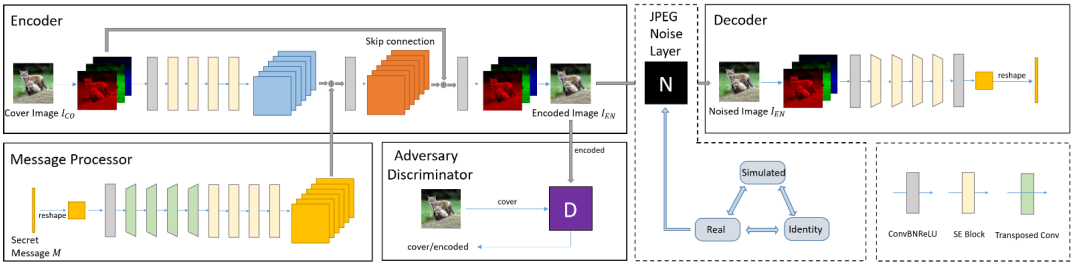

图 11.7　MBRS 的架构。图摘自原论文

以下，将对架构的这些部分做简要介绍：

消息处理器：它的作用是将带嵌入的消息处理为特征图，以极大增强嵌入消息的鲁棒性。具体来说，对于待嵌入的长度为 $L = 2^{2N}$（如 $L = 2^6 = 64$）的二进制消息 $M \in \{0,1\}^L$，它将先被重塑（reshape）为 $1 \times 2^N \times 2^N$（如 $1 \times 8 \times 8$）的张量（实际即为 $2^N \times 2^N$ 的矩阵），接着通过带有批归一化（Batch Normalization）和 ReLU 的卷积层（见架构图右下角的 ConvBNReLU），再通过四次转置卷积层（见架构图右下角的 Transposed Conv）放大为和输入图片一般大小的特征图，最后通过四层 SE Block 筛选不易被压缩掉的区域，在这些区域隐藏信息。

编码器和解码器：它们的网络结构主要是 SE Block 和卷积层，编码器将图片编码为特征图后，与消息处理器得到的特征图相加，得到最终的特征图。在这之后，引入原图的色彩信息，便可以将隐藏了信息的特征图嵌入到图像中。

JPEG 噪声层：即前述提到的带有混合切换逻辑的噪声层。

对抗辨别器：参考了生成对抗网络（Generative Adversarial Network，GAN）的思想，它将负责甄别原图（cover image）和编码图（encoded image），并借由这种对抗的过程，迫使编码器迭代优化，最终生成“以假乱真”的编码图。这确保了数字水印不会影响原图片的正常使用。

扩散块：面对裁剪攻击的对策

在没有扩散块的深度学习水印模型中，信息的嵌入通常是局部化的。由于编码器主要由卷积层组成，卷积层只能看到局部的小窗口。例如，消息序列中的第 1 位可能只影响图片左上角的一块像素；第 64 位只影响右下角。如果攻击者裁剪掉了图片的左上角，那么第 1 位的信息就彻底消失了。无论解码器多强，都无法找回已经不存在的像素。扩散块（Diffusion Block）的出现，就是为了打破这种对应关系，让每一位的信息都弥散到整张图片的每一个像素中。

如图 11.8 所示，扩散块的原理十分简单，即让消息在进入消息处理器之前，先经过全连接层，输出一个相同长度的向量，再将这个向量送入到消息处理器中。这时，向量中的每一个元素都包含了原始消息中每一位的混合结果。而在解码器的末端，同样需要一个全连接层，用以执行逆向的操作，将弥散的信息重新汇聚回原始的序列。

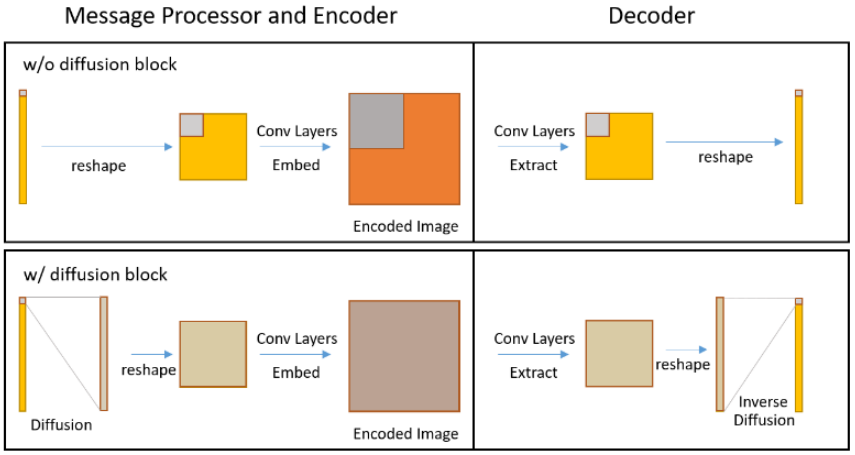

图 11.8　扩散块的架构。图摘自原论文

虽然扩散块很强，但它也有严重的缺陷：全连接层的参数量随输入输出大小呈平方级增长（$O(N^2)$）。对于超大分辨率图片，直接使用全局全连接层会导致内存爆炸。因此，MBRS 通常只在低分辨率的特征层进行扩散；此外，由于信息被洒在了全图，可能会在整张图片上产生轻微的、连贯的噪点纹理，相比局部嵌入，它更难保持完全不可见。因此，在 MBSR 中，它被描述为是一个”补充”组件。

损失函数

为了让模型既能生成清晰的图像,又能使它们具有抗压缩的强鲁棒性,需要定义复杂的损失函数。

1. **图像质量损失**(L_{E1}):

$$L_{E1} = MSE(I_{CO}, I_{EN}) = MSE(I_{CO}, E(\theta_E, I_{CO}, M))$$

其中 I_{CO} 是原图(cover image),I_{EN} 是嵌入水印后的图(encoded image)。$MSE(\mathbf{x}, \mathbf{y}) \equiv ||\mathbf{x} - \mathbf{y}||_2^2$ 为均方误差。$E(\cdot)$ 即编码器网络所表征的映射,θ_E 为编码器参数。目的是让两者尽可能像。

2. **解码损失**(L_D)

$$L_D = MSE(M, M') - MSE(M, D(\theta_D, I_{NO}))$$

其中 M 是原始消息,M' 是提取的消息。$D(\cdot)$ 即解码器网络所表征的映射,θ_D 为解码器参数。目的是让误码率降到最低。

3. **对抗损失**(L_{E2} 和 L_A):

$$L_A = \log(1 - A(\theta_A, E(\theta_E, I_{CO}, M)) + \log(A(\theta_A, I_{CO})))$$
$$L_{E_2} = \log(A(\theta_A, I_{EN})) = \log(A(\theta_A, E(\theta_E, I_{CO}, M)))$$

其中 $A(\cdot)$ 即辨别器网络所表征的映射,θ_A 为辨别器参数。目的是提升辨别器的甄别能力,同时通过对抗(即优化 L_{E_2})迫使编码器生成更高质量的图像。

最终定义总的损失函数如下

$$L_{encoder,\ decoder} \equiv \lambda_E L_{E_1} + \lambda_D L_D + \lambda_A L_{E_2}$$
$$L_{discriminator} \equiv L_A$$

11.3 实验环境配置

11.3.1 Anaconda 的安装

根据附录 A.1 Python 实验环境配置 中 Anaconda 的安装与配置部分,安装适配当前系统的 Anaconda 环境。

安装完成后打开命令行(`Win+R`,输入 `cmd`,打开命令行)并输入

```
conda -version
```

若返回版本号,如 `conda 23.7.4`,则说明 Anaconda 已经正确安装并注册入环境变量 `PATH` 中。

11.3.2　源代码下载与环境配置

原论文已经提供了 Github 仓库地址:https://github.com/jzyustc/MBRS,可使用 git clone 指令拉取源代码

```
git clone https://github.com/jzyustc/MBRS.git
```

若无法使用 git clone 拉取,也可选择 Code → Download Zip 下载 Zip 压缩的源代码后解压。

在本机上的环境配置

拉取仓库后,进入 MBRS 目录,创建名为 requirements.txt 的文本文件,并写入依赖包名称和版本号

```
# Linux 环境下; Windows 环境直接使用文本编辑器即可
cd MBRS
touch requirements.txt
nano requirements.txt
```

```
### 向 requirements.txt 中写入以下内容 ###

# 指定 PyTorch 官方仓库优先以下载 CUDA 支持的 PyTorch, 这里选定 CUDA 版本为
    10.1
--index-url https://download.pytorch.org/whl/cu101

# 指定清华源作为额外源, 清华源只能下载到 CPU 版本的 PyTorch
--extra-index-url https://pypi.tuna.tsinghua.edu.cn/simple

numpy==1.16.4
Pillow==6.0.0
scipy==1.3.0
kornia==0.3.0
torch==1.5.0+cu101
torchvision==0.6.0+cu101
```

接着，为 MBRS 创建虚拟环境。由于 MBRS 的实现基于较早版本的 Pytorch，建议下载较早版本的 Python，如 Python 3.7：

```
conda create -n MBRS_env python=3.7 -y
conda activate MBRS_env
```

通过 `requirements.txt` 为环境安装依赖

```
(MBRS_env) pip install -r requirements.txt
conda activate MBRS_env
```

等待安装结束，环境配置完成。

在 SCOW 超算平台上的环境配置

若在 SCOW 超算平台上进行操作，则因为平台提供算力基于 RTX 5090 集群，其显卡架构仅支持高版本的 Python 和 Torch：

```
# 经测试，Nightly 版本的 Torch 在 Python 3.11 下可以正常安装
conda create -n MBRS_env python=3.11 -y
conda activate MBRS_env
```

接着，安装 Nightly 版本的 Torch，目前只有它提供了对 5090 架构的支持：

```
(MBRS_env) pip install --pre torchvision --index-url https://download.
    pytorch.org/whl/nightly/cu124
```

再安装 MBRS 所需的其它依赖：

```
(MBRS_env) pip install numpy scipy Pillow kornia
```

等待安装结束，环境依赖配置完成。由于 MBRS 使用的是较早版本的 Python 和其它依赖包，在较新的环境下，需要对部分代码进行修改。具体来说，需要修改其 `network` 目录下的 `network/Network.py` 的关于 `kornia` 库的代码：

```
# 第 102 行左右，原代码
# ssim
# ssim = 1 - 2 * kornia.losses.ssim(encoded_images.detach(), images,
    window_size=5, reduction="mean")

# 修改为如下
# 对第 146 行左右和第 208 行左右的相同代码做同样处理
ssim = 1 - 2 * kornia.losses.ssim_loss(encoded_images.detach(), images,
    window_size=5, reduction="mean")
```

11.4　数据集准备和说明

本实验中会使用到 ImageNet 和 COCO 数据集。

11.4.1　ImageNet 数据集

ImageNet 是由斯坦福大学的李飞飞教授团队在 2009 年启动的数据集，其包含超过 1400 万张手动标注的图片，并根据 WordNet（一个英语词汇分级数据库）组织建立。

由于训练集过大，且原论文仅使用了 10000 张图片用作训练，为了方便和节约时间，本次实验将使用 ImageNet 的其中一个子数据集 ImageNet ILSVRC 2016 中的测试集作为训练数据，其包含了 8706 张图片，已经上传至云盘可供下载：https://pan.ustc.edu.cn/share/index/4e2e2fba45064f89a426

11.4.2　COCO 数据集

COCO 由微软于 2014 年发布，全称是“常见上下文中的物体”（Common Objects in Context）。其相较于 ImageNet 数据集，更注重图片所在的环境（即语境、上下文）

本实验将使用 COCO 2014 Val Images 中的 1000 张作为验证集，1000 张作为测试集，已经上传至云盘可供下载：https://pan.ustc.edu.cn/share/index/1f0f90c493a34ebeb901

11.4.3　加载数据集

在拉取仓库的根目录 `MBRS` 下创建 `datasets` 目录，在该目录下再创建 `train`、`test`、`validation` 三个文件夹。

```
# Linux 环境下
touch datasets
cd datasets
mkdir train, test, validation
```

将云盘中下载的压缩包 train.zip、validation.zip、test.zip 分别放入对应的目录中,并解压

```
# Linux 环境下
# 假设压缩包现置于 /ghome/gpub/MBRS_dataset 中，而仓库位置在 /ghome/MBRS 下

# 复制压缩包至对应目录
cp /ghome/gpub/MBRS_dataset/train.zip /ghome/MBRS/datasets/train
cp /ghome/gpub/MBRS_dataset/validation.zip /ghome/MBRS/datasets/validation
cp /ghome/gpub/MBRS_dataset/test.zip /ghome/MBRS/datasets/test

# 到达对应目录并解压
cd /ghome/MBRS/datasets/train
unzip train.zip
cd /ghome/MBRS/datasets/validation
unzip validation.zip
cd /ghome/MBRS/datasets/test
unzip test.zip
```

执行完毕后，目录 datasets 的结构应该如图 11.9

```
├── datasets
│   ├── train
│   │   ├── xxx.jpg
│   │   ├── ...
│   ├── test
│   │   ├── xxx.jpg
│   │   ├── ...
│   ├── validation
│   │   ├── xxx.jpg
│   │   ├── ...
├── network
├── results
├── ...
```

图 11.9　datasets 的目录树

11.5　实验内容

11.5.1　实验一：源代码验证

第一个实验的内容是尝试执行训练代码。预期通过该过程学习上手使用 SCOW 超算平台并熟悉源代码。

参考 SCOW 学生使用手册的内容提交作业并执行 train.py 脚本以开始训练。以下将对该脚本及其涉及的代码做简要解释。train.py 的工作流如下，它反映了深度学习模型训练所遵循的一般步骤：

1. **参数初始化**。通过 utils 目录下定义的 load_train_setting.py 读取训练脚本同目录下 train_setting.json 配置文件中的参数设置。
2. **数据集准备**。通过 utils 目录下定义的数据集类 MBRSDataset、torch 定义的数据集加载器 DataLoader 来读取训练集和验证集数据。
3. **模型实例化**。实例化在 network 中定义的 Network 类，其封装了 MBRS 的整体架构，包含编码器、解码器和判别器；此外，若在配置文件中开启了续练功能 train_continue，则会读取 results 文件中训练过的模型（路径同样在 train_setting.json 中定义）
4. **主训练循环**。包含 train 和 validation 两部分，调用 Network 类中定义的 train 与 validation 函数，并保存结果写入日志。
5. **保存模型**。保存模型。在每个 epoch 结束后，将模型保存为 .pth 文件。

训练结束后，可在 results 文件中找到结果。其命名格式为项目名 + 消息长度 + 噪声层类型 + 训练开始时间。一个示例如下：

MBRS_m64_Combined([JpegMask(50),Jpeg(50),Identity()])__2026_02_18__10_28_06

将其重命名为易于访问的名称，如 MBRS_test 方便后续测试。在与 train.py 同目录下的 test_settings.json 中修改模型路径 result_folder：

```
{
    "result_folder": "MBRS_test",
    ...,
}
```

之后，通过作业提交功能执行 test.py 以进行测试。test.py 的工作流如下：

1. **参数加载**。通过 utils 目录下定义的 load_test_setting.py 读取训练脚本同目录下 test_setting.json 配置文件中的参数设置。
2. **权重加载**。通过 Network 类定义的 load_model_ed 函数加载指定目录下的权重文件。
3. **数据集加载**。通过 utils 目录下定义的数据集类 MBRSDataset、torch 定义的数据集加载器 DataLoader 来读取测试集数据。
4. **主循环**。其主要工作是，由编码器生成水印，并应用强度因子（Strength Factor）进行缩放。之后，通过噪声层执行压缩，最后经过解码器来尝试恢复消息的二进制序列。
5. **指标汇总**。训练和测试时，主要依赖以下三个评估指标：
   (a) **误码率（Bit Error Rate, BER）**。表达式为

$$BER \equiv \frac{1}{L}\sum_{i=1}^{L} 1_{\{M_i \neq M_i'\}}$$

   其直接评估了原信息和还原后的信息每一比特位的差别。
   (b) **PSNR（Peak Signal-to-Noise Ratio）**。其衡量了图像像素值的物理保真度。其 > 35 dB 时，说明图像保真度良好，不会影响使用。
   (c) **SSIM（Structural Similarity）**。其衡量了图像结构的感知相似度。相比 PSNR，其更贴近人眼的视觉主观感受。越接近 1.0，说明嵌入后的图和原图的主观感受差异越小，保真度越高。

11.5.2 实验二：裁剪攻击与扩散块测试

第二个实验的内容是通过修改配置文件以引入扩散层，并使用裁剪攻击以验证扩散层的性能。预期通过该实验加深对扩散层及其涉及的卷积神经网络的“感受野”概念的理解。

找到与训练脚本同目录下的配置文件 train_settings.json，将 with_diffusion 设置为 true，并将 noise_layers 修改为

```
// 原 noise_layers
// "Combined([JpegMask(50),Jpeg(50),Identity()])"
// 修改为
"Combined([JpegMask(50),Jpeg(50),Identity(),Crop(0.15,0.15)])"
```

参考 SCOW 学生使用手册的内容提交作业，执行 `train.py` 脚本以开始训练。分别记录在开启扩散层和关闭扩散层后的训练指标与图示，并通过作业提交功能执行 test.py 以测试结果，重点关注训练结果目录中 `images` 目录下的内容，它展示了部分训练数据嵌入水印或经过噪声攻击后的图示，此外还有残差信号的图示。对于加入了扩散层的 MBRS，其残差图的纹理分布是较为分散的，如此便能有效对抗裁剪攻击。

一个可能的示例训练结果如图 11.10 所示，其中右侧两张图片在随机噪声层中抽取到了裁剪攻击层 `Crop(0.15, 0.15)`（裁剪比例为 0.15 的裁剪攻击）：

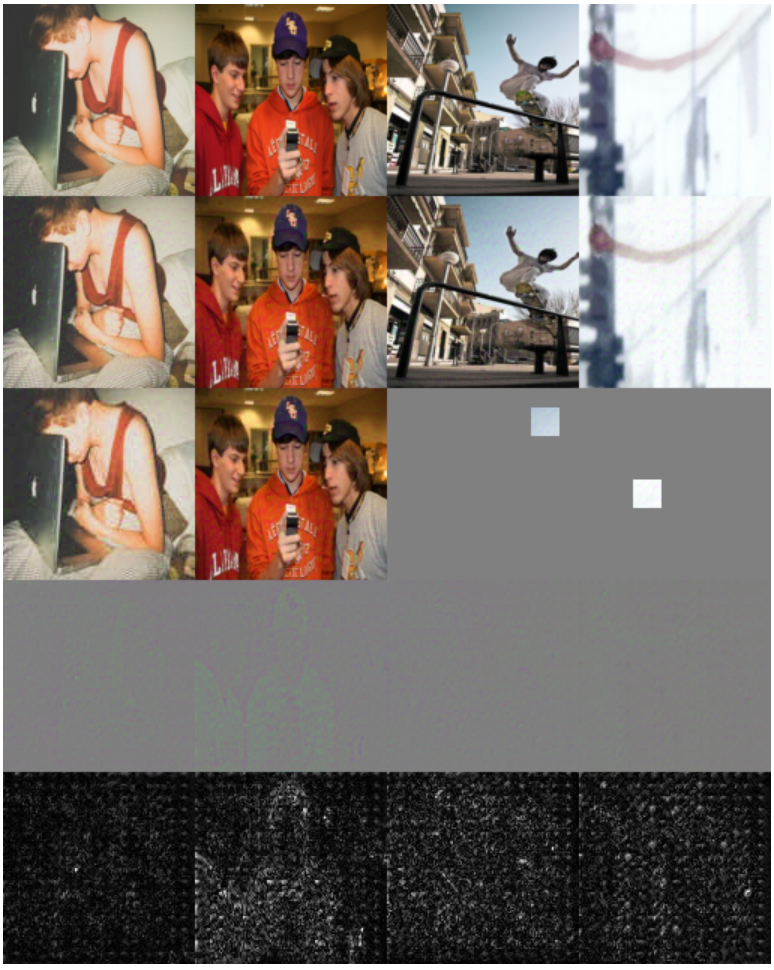

图 11.10　加入扩散层训练 100 个 epoch 后的结果

11.5.3 实验三:通道注意力的改进

第三个实验的内容将涉及实际的代码修改。目标是借鉴 Transformer 中自注意力机制的设计思想,对 MBRS 原始的通道注意力(SE Block)进行改进。预期通过该实验,使学生深入掌握注意力机制从“静态加权”向“动态关联建模”跨越的核心逻辑。

改进的动机在于:原始 SE 模块仅通过全连接层对压缩后的全局特征进行映射,其本质是学习一种固定的通道重要性分布,缺乏对通道间复杂关联的动态捕捉。引入 Transformer 的自注意力机制后,模型可以通过计算查询(Query)与键(Key)的相似度,显式地建模不同特征通道间的语义相关性。这种机制能赋予模型更强的容错能力:即使某些特征通道在 JPEG 压缩中遭受了严重的量化破坏,解码器仍能通过与其高度相关的其他通道中的残余信号进行特征补偿,从而在极端噪声环境下实现更稳健的水印提取。

该实验将要修改或创建的内容分别位于:

- `networks/blocks` 目录下将创建的新模块 `SelfAttentionSENet.py`;
- `networks` 目录下的 `Encoder_MP_Decoder.py`、`Encoder_MP.py`、`Network.py`、`Decoder.py`;
- `utils` 目录下的 `load_test_setting.py`、`load_train_setting.py`;
- 根目录下的配置文件 `train_settings.json`、`test_settings.json`;

SelfAttentionSENet 模块的定义

`networks/blocks` 目录存放了 MBRS 架构中所包含的模块。将仿照 `SENet.py` 的格式,定义新的模块 `SelfAttentionSENet.py` 来改进通道注意力。

以下,将对 `SENet.py` 的代码结构做解析。它的宏观架构实际是 ResNet 残差神经网络的变体,共定义了四个类:

- `BasicBlock`:由两个 3×3 的卷积组成,用于处理特征复杂度较低的早期层。
- `BottleneckBlock`:由两个 1×1 和一个 3×3 的卷积层组成,第一个 1×1 卷积负责将通道数压缩(降维),3×3 卷积负责在压缩后的空间内高效提取特征;其后 1×1 的卷积将升维以恢复通道数。
- `SENet`:用于编码器的网络封装结构,其进行特征增强,但不改变图片的尺寸。
- `SENet_decoder`:用于解码器的网络封装结构,其进行空间压缩,将分散在像素中的信息不断汇聚到特征维度中,最终坍缩为特征向量。

SE Block 相较于 ResNet 的修改在于,它在 `BasicBlock` 和 `BottleneckBlock` 中定义了 `se` 网络(`self.se`),跟随在原先的特征提取网络 `self.left` 之后,其功能在两个块中是相同的,以 `BasicBlock` 类中的 `self.se` 为例,其代码解析如下:

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, r, drop_rate):
        super(BasicBlock, self).__init__()
```

```python

        # ...

        self.se = nn.Sequential(
            # Step 1: 挤压（Squeeze）
            # 将 H * W 的空间维度压缩为通道维度
            nn.AdaptiveAvgPool2d((1, 1)),

            # Step 2: 两个 1 * 1 卷积
            # 利用全连接的性质，学习不同通道间的权重关系
            nn.Conv2d(in_channels=out_channels, out_channels=out_channels //
                r, kernel_size=1, bias=False),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_channels=out_channels // r, out_channels=
                out_channels, kernel_size=1, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        identity = x
        x = self.left(x)
        scale = self.se(x)

        # Step 3: 缩放（Scale）
        # 将学习到的权重乘回原始的特征图，完成加权
        x = x * scale

        if self.downsample is not None:
            identity = self.downsample(identity)

        x += identity
        x = F.relu(x)
        return x
```

本实验对 SENet.py 的主要修改也集中于对 self.se 的修改。具体来说，将定义新的通道注意力模块类 ChannelSelfAttention，并用其代替 self.se 中的网络结构：

```
self.se = ChannelSelfAttention(out_channels, r=r)
```

以下为具体的步骤：

1. 在 blocks 目录下创建 SelfAttentionSENet.py,并复制 SENet.py 的内容。
2. 在 SelfAttentionSENet.py 中定义类 ChannelSelfAttention，框架代码已给出如下。需要回忆自注意力权重的公式：

$$Attention(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

```
# 通道自注意力模块
class ChannelSelfAttention(nn.Module):
    # 借鉴 Transformer 思想的通道自注意力模块

    def __init__(self, channels, r=8, heads=4):
        super(ChannelSelfAttention, self).__init__()
        self.heads = heads

        # 将计算后的中间维度保存为成员变量
        self.inter_channels = channels // r
        self.head_dim = self.inter_channels // heads

        self.avg_pool = nn.AdaptiveAvgPool2d(1)

        # 线性映射层：一次性生成 Q, K, V 的原始张量
        self.to_qkv = nn.Linear(channels, self.inter_channels * 2 +
            channels, bias=False)

        self.scale = self.head_dim ** -0.5

        self.fc = nn.Linear(channels, channels)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        b, c, h, w = x.shape

        # 1. Squeeze 操作
```

```
        y = self.avg_pool(x).view(b, c)

        # 2. 生成 Q, K, V
        qkv = self.to_qkv(y)

        # ---------- 此为 TODO 1 开始 ----------

        # TODO: 利用 torch.split 将 qkv 划分为 q_raw, k_raw, v_raw
        # HINT: 长度分配为 [self.inter_channels, self.inter_channels, c
            ]

        # ---------- 此为 TODO 1 结束 ----------

        # 重塑为多头注意力格式（为了进行多头注意力计算）
        # q, k 形状: (B, heads, 1, head_dim)
        q = q_raw.view(b, self.heads, 1, self.head_dim)
        k = k_raw.view(b, self.heads, 1, self.head_dim)
        # v 形状: (B, heads, 1, c // heads)
        v = v_raw.view(b, self.heads, 1, -1)

        # ---------- 此为 TODO 2 开始 ----------

        # TODO: 计算缩放点积注意力（Scaled Dot-Product Attention）
        # HINT: 已给出注意力公式, 注意使用 torch.matmal, self.scale (即
            1/sqrt{d_k}) 和 F.softmax

        # ---------- 此为 TODO 2 结束 ----------

        # 4. 融合 Value 并还原维度
        # out 形状: (B, C)
        out = torch.matmul(attn, v).view(b, c)

        # ---------- 此为 TODO 3 开始 ----------

        # TODO: 生成最终的通道权重 scale 以作用于 x
        # HINT: 经过 self.fc 和 self.sigmoid, 需注意形状对齐 (b, c, 1,
```

```
            1), 可使用 .view

        # ---------- 此为 TODO 3 结束 ----------

        return x * scale
```

3. 将 BasicBlock 和 BottleneckBlock 中的 self.se 修改为上文给出的代码。(建议单独定义它们的类名,如 SelfAttnBasicBlock 以防止在后续引用时出现重名问题)

Network 的修改

本部分的修改旨在让原代码的编码器和解码器能够适配新定义的模块。为此,需要仿照原代码的格式定义新的类。

对 Encoder_MP.py 的修改。定义新的类 Encoder_MP_SelfAttn,让其使用新定义的通道自注意力。

```
class Encoder_MP_SelfAttn(nn.Module):
    # 使用 Transformer 通道注意力的编码器

    def __init__(self, H, W, message_length, blocks=4, channels=64):
        super(Encoder_MP_SelfAttn, self).__init__()
        self.H = H
        self.W = W

        message_convT_blocks = int(np.log2(H // int(np.sqrt(message_length)
            )))
        message_se_blocks = max(blocks - message_convT_blocks, 1)

        self.image_pre_layer = ConvBNRelu(3, channels)

        # ---------- 此为 TODO 1 开始 ----------
        # TODO: 将图像特征提取修改为 SelfAttentionSENet
        # HINT: 参考原代码中该部分的内容, 注意指定 block_type 为
            BottleneckBlock; 此后若无明确指出, 都需指定 block_type

        # ---------- 此为 TODO 1 结束 ----------

```

```
20          self.message_pre_layer = nn.Sequential(
21              ConvBNRelu(1, channels),
22              ExpandNet(channels, channels, blocks=message_convT_blocks),
23              # ---------- 此为 TODO 2 开始 ----------
24              # TODO: 将消息预处理修改为 SelfAttentionSENet
25              # HINT: 参考原代码中该部分的内容，注意 blocks 的不同
26
27              # ---------- 此为 TODO 2 结束 ----------
28          )
29
30          # ---------- 此为 TODO 3 开始 ----------
31          # TODO: 将消息主特征层修改为 SelfAttentionSENet
32          # HINT: 参考原代码中该部分的内容
33
34          # ---------- 此为 TODO 3 结束 ----------
35
36          self.after_concat_layer = ConvBNRelu(2 * channels, channels)
37          self.final_layer = nn.Conv2d(channels + 3, 3, kernel_size=1)
38
39      def forward(self, image, message):
40          # 代码保持不变，但内部算子已升级
41          image_pre = self.image_pre_layer(image)
42          intermediate1 = self.image_first_layer(image_pre)
43
44          size = int(np.sqrt(message.shape[1]))
45          message_image = message.view(-1, 1, size, size)
46          message_pre = self.message_pre_layer(message_image)
47          intermediate2 = self.message_first_layer(message_pre)
48
49          concat1 = torch.cat([intermediate1, intermediate2], dim=1)
50          intermediate3 = self.after_concat_layer(concat1)
51          concat2 = torch.cat([intermediate3, image], dim=1)
52          output = self.final_layer(concat2)
53          return output
```

对 Decoder.py 的修改。定义新的类 Decoder_SelfAttn,引入自定义的通道自注意力。

```python
class Decoder_SelfAttn(nn.Module):
    # 使用 Transformer 通道注意力的解码器

    def __init__(self, H, W, message_length, blocks=4, channels=64):
        super(Decoder_SelfAttn, self).__init__()

        stride_blocks = int(np.log2(H // int(np.sqrt(message_length))))
        keep_blocks = max(blocks - stride_blocks, 0)

        self.first_layers = nn.Sequential(
            ConvBNRelu(3, channels),
            # ---------- 此为 TODO 1 开始 ----------
            # TODO: 使用 SelfAttentionSENet_decoder 来处理下采样
            # HINT: 参考原代码中该部分的内容, 注意 blocks 的设置

            # ---------- 此为 TODO 1 结束 ----------
            ConvBNRelu(channels * (2 ** stride_blocks), channels),
        )

        # ---------- 此为 TODO 2 开始 ----------
        # TODO: 将保持层修改为 SelfAttentionSENet
        # HINT: 参考原代码中该部分的内容, 注意 blocks 的设置

        # ---------- 此为 TODO 2 结束 ----------

        self.final_layer = ConvBNRelu(channels, 1)

    def forward(self, noised_image):
        x = self.first_layers(noised_image)
        x = self.keep_layers(x)
        x = self.final_layer(x)
        x = x.view(x.shape[0], -1)
        return x
```

对 Encoder_MP_Decoder.py 的修改。仿照 EncoderDecoder 的结构，定义新的编码解码器封装类

```
class EncoderDecoder_SelfAttn(nn.Module):
    # 全系统架构: Encoder(SelfAttn) -> Noise -> Decoder(SelfAttn)

    def __init__(self, H, W, message_length, noise_layers):
        super(EncoderDecoder_SelfAttn, self).__init__()

        # ---------- 此为 TODO 1 开始 ----------
        # TODO: 给出 self.encoder, self.noise, self.decoder 的定义
        # HINT: 参考原代码中该部分的内容

        # ---------- 此为 TODO 1 结束 ----------

    def forward(self, image, message):
        encoded_image = self.encoder(image, message)
        noised_image = self.noise([encoded_image, image])
        decoded_message = self.decoder(noised_image)
        return encoded_image, noised_image, decoded_message
```

对 network.py 的修改。为 Network 类添加新的配置参数 with_self_attention 并重写网络架构选择逻辑。

```
# ---------- 待修改的代码开始 ----------
# TODO: 重写网络架构选择逻辑，使其支持三种架构: with_diffusion（扩散块），
    with_self_attention（自通道注意力），naive（原始）

if not with_diffusion:
    self.encoder_decoder = EncoderDecoder(H, W, message_length,
        noise_layers).to(device)
else:
    self.encoder_decoder = EncoderDecoder_Diffusion(H, W, message_length,
        noise_layers).to(device)
# ---------- 待修改的代码结束 ----------
```

配置文件与配置加载脚本的修改

对配置文件的修改。在 `train_settings.json` 和 `test_settings.json` 中添加项 `with_self_attention`,取值自定。

对配置加载脚本的修改。在 `utils` 目录中的 `load_train_setting.py` 和 `load_test_setting.py` 中定义变量 `with_self_attention`,具体参照原代码内容。

训练与测试

至此,对通道注意力的改进已经完成,可以通过 `train_settings.json` 和 `test_settings.json` 自由配置是否要在训练和测试中使用自定义的通道注意力。

参考 SCOW 学生使用手册的内容提交作业,**关闭裁剪攻击后**,执行 `train.py` 脚本以开始训练。分别记录在启用和关闭自定义通道注意力,且不引入扩散块的情况下,二者的实验指标和图示结果。重点关注两种情形下的图示结果(即 `images` 目录下的结果)中残差信号图的差别。分析引起这种差别的可能原因。

11.6 实验要求与评分细则

11.6.1 需要提交的内容

1. 一份实验报告,**只需**包含三次实验的训练与测试结果,实验指标与图示,以及思考题的解答。
2. 实验三中涉及修改的代码文件,将其与实验报告一同打包发送。

11.6.2 评分细则

1. (20%) 完成实验一,并且得到的测试指标处于合理范围:

$$BER(error\ rate) \leqslant 0.8\%, \quad PSNR \geqslant 32\text{dB}, \quad SSIM \geqslant 0.85$$

2. (20%) 完成实验二,得到的结果应当合理,即添加扩散块的结果应该优于未添加扩散块的结果。
3. (30%) 完成实验三的代码修改与训练测试,代码完整且正确。
4. (30%) 思考题。须有详尽的解答,可以使用 AI 工具辅助理解。

11.7 思考题

1. 通过该实验的学习,总结对于全零梯度的损失函数,可能的解决方案有哪些?

2. 回忆动量优化器如 Adam Optimizer 的原理。当噪声层随机选择到了原始 JPEG 压缩时，优化器在先前模拟 JPEG 或恒等映射中累计的动量是如何被用来更新编码器参数的？
3. 尝试对比 SE Block 的通道注意力和 Transformer 的自注意力的差别。例如，它们的基本作用单元，它们是如何添加注意力的。
4. 尝试从卷积神经网络的感受野角度分析，为什么裁剪攻击对不使用扩散块的水印系统是致命的。
5. 尝试分析使用了自定义的通道自注意力后，其残差图结果与使用 SE Block 的通道注意力的差别，并分析可能的原因。（无标准答案，言之有理即可）